
Bölüm-5

STRING, KARAKTERLER VE DÜZENLİ İFADELER

OBJECTIVES



Bu bölümde;

- ▶ **String sınıfının değişmez karakter dizisi nesnelerini oluşturacaksınız.**
- ▶ **split yöntemini kullanarak bir String nesnesini ayıracaksınız.**
- ▶ **Bir uygulamaya girilen String verilerini doğrulamak için düzenli ifadeler kullanacaksınız.**
- ▶ **StringBuilder sınıfının değiştirilebilir karakter dizisi nesneleri oluşturacaksınız.**
- ▶ **Character sınıfının nesnelerini oluşturacaksınız.**

OUTLINE



- ▶ Giriş
- ▶ Character ve String Temelleri
- ▶ String Sınıfı
- ▶ String Tokenizing
- ▶ Düzenli İfadeler
- ▶ StringBuilder Sınıfı
- ▶ Character Sınıfı



- ▶ Burada verilen teknikler,
 - ▶ program girişini doğrulamak
 - ▶ kullanıcılara bilgi göstermek
 - ▶ Diğer metin tabanlı işlemleri gerçekleştirmek için uygundur.
 - ▶ Ayrıca metin düzenleyiciler, kelime işlemciler, sayfa düzeni yazılımı ve diğer metin işleme yazılımları geliştirmek için de uygundurlar.
- ▶ Uygulamalara girdiyi doğrulama yeteneği sağlayan düzenli ifadeler de anlatılacaktır.

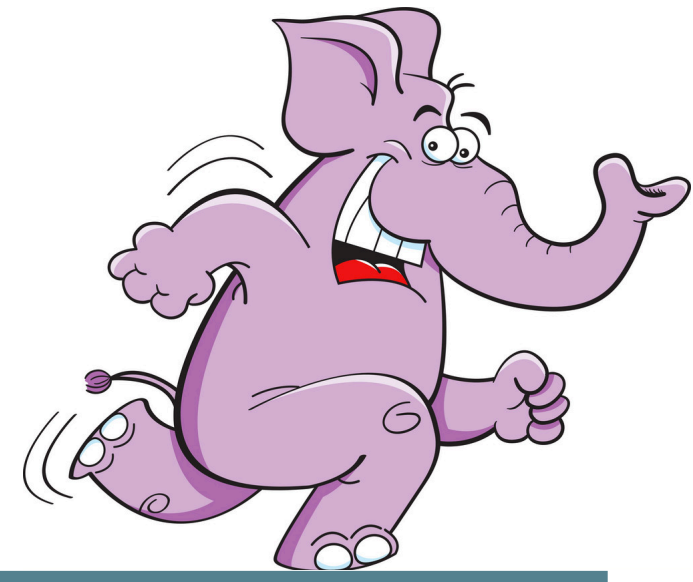


- ▶ Karakterler, Java programlarının temel yapı taşlarıdır.
- ▶ Her program, anlamlı bir şekilde gruplandırıldığında Java derleyicisi tarafından bir görevi gerçekleştirmek için kullanılan bir dizi talimat olarak yorumlanan bir dizi karakterden oluşur.
- ▶ Bir program karakter değişmezleri içerebilir.
- ▶ Bir karakter değişmezi, tek tırnak içinde bir karakter olarak temsil edilen bir tamsayı değeridir. Örneğin;
 - ▶ 'z', z'nin tamsayı değerini temsil eder.
 - ▶ '\t', bir sekme karakterinin tamsayı değerini temsil eder.
 - ▶ Bir karakter değişmezinin değeri, Unicode karakter kümesindeki karakterin tamsayı değeridir (ASCII).
 - ▶ Ek B, Unicode'un bir alt kümesi olan ASCII karakter kümesindeki karakterlerin tamsayı eşdeğerleri bulunmaktadır.



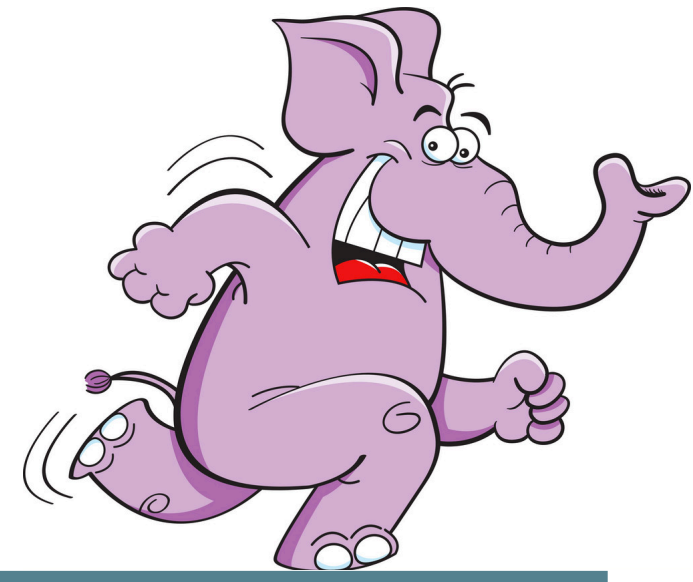
- ▶ String tek bir birim olarak ele alınan bir dizi karakterden oluşur.
 - ▶ Harfler, rakamlar ve +, -, *, / ve \$ gibi çeşitli özel karakterler içerebilir.
 - ▶ Bir string, String sınıfının bir nesnesidir.
 - ▶ String sabitleri (string literals) bellekte String nesneleri olarak depolanır.
 - ▶ Çift tırnak içinde bir dizi karakter olarak yazılır;
 - ▶ "John Q. Doe" (a name)
 - ▶ "9999 Main Street" (a street address)
 - ▶ Bir string referansında da tutulabilir.
 - ▶ String color = "blue";

PERFORMANCE TIP



Belleği korumak için Java, aynı içeriğe sahip tüm string sabitlerini kendisine birçok referansı olan tek bir String nesnesi olarak ele alır.

PERFORMANCE TIP



Java SE 9'dan itibaren Java, daha kompakt bir String temsili kullanır.

Bu, yalnızca Latin-1 karakterlerini (0-255 karakter kodlarına sahip Stringleri) depolamak için kullanılan bellek miktarını önemli ölçüde azaltır.



► String oluşturma'nın farklı şekilleri vardır.

```
1 // Fig. 14.1: StringConstructors.java
2 // String class constructors.
3
4 public class StringConstructors {
5     public static void main(String[] args) {
6         char[] charArray = {'b', 'i', 'r', 't', 'h', ' ', 'd', 'a', 'y'};
7         String s = new String("hello");
8
9         // use String constructors
10        String s1 = new String();
11        String s2 = new String(s);
12        String s3 = new String(charArray);
13        String s4 = new String(charArray, 6, 3);
14
15        System.out.printf(
16            "s1 = %s\ns2 = %s\ns3 = %s\ns4 = %s\n", s1, s2, s3, s4);
17    }
18 }
```

```
s1 =
s2 = hello
s3 = birth day
s4 = day
```

Fig. 14.1 | String class constructors.



- ▶ `length()`
- ▶ `charAt()`
- ▶ `getChars()`

```
1 // Fig. 14.2: StringMiscellaneous.java
2 // This application demonstrates the length, charAt and getChars
3 // methods of the String class.
4
5 public class StringMiscellaneous {
6     public static void main(String[] args) {
7         String s1 = "hello there";
8         char[] charArray = new char[5];
9
10        System.out.printf("s1: %s", s1);
11
12        // test length method
13        System.out.printf("%nLength of s1: %d", s1.length());
14
15        // loop through characters in s1 with charAt and display reversed
16        System.out.printf("%nThe string reversed is: ");
17
18        for (int count = s1.length() - 1; count >= 0; count--) {
19            System.out.printf("%c ", s1.charAt(count));
20        }
```

Fig. 14.2 | String methods length, charAt and getChars. (Part 1 of 2.)


```
21
22     // copy characters from string into charArray
23     s1.getChars(0, 5, charArray, 0);
24     System.out.printf("%nThe character array is: ");
25
26     for (char character : charArray) {
27         System.out.print(character);
28     }
29
30     System.out.println();
31 }
32 }
```

```
s1: hello there
Length of s1: 11
The string reversed is: e r e h t   o l l e h
The character array is: hello
```

Fig. 14.2 | String methods length, charAt and getChars. (Part 2 of 2.)



▶ String karşılaştırma

▶ Alfabedeki harflerin öncelikleri bilinir. B harfi J harfinden önce gelir.

▶ PEKi bilgisayar bunu nasıl bilecek?

▶ String karşılaştırma yaparken, aslında numerik değerler karşılaştırılır.

▶ equals()

▶ == operator

▶ equalsIgnoreCase()

▶ compareTo()

▶ regionMatches()

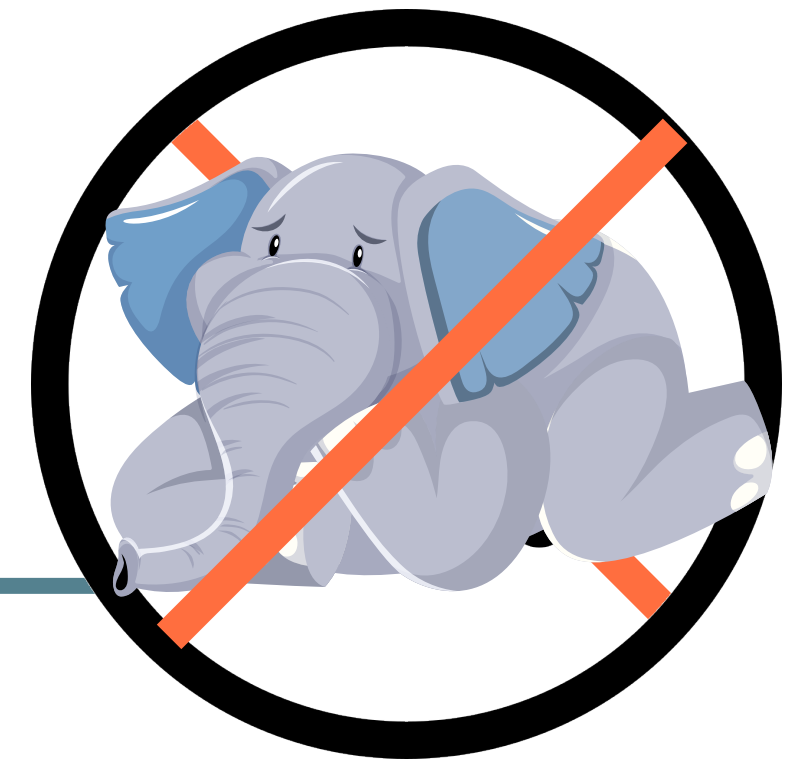
```
1 // Fig. 14.3: StringCompare.java
2 // String methods equals, equalsIgnoreCase, compareTo and regionMatches.
3
4 public class StringCompare {
5     public static void main(String[] args) {
6         String s1 = new String("hello"); // s1 is a copy of "hello"
7         String s2 = "goodbye";
8         String s3 = "Happy Birthday";
9         String s4 = "happy birthday";
10
11         System.out.printf(
12             "s1 = %s\ns2 = %s\ns3 = %s\ns4 = %s\n\n", s1, s2, s3, s4);
13
```

Fig. 14.3 | String methods equals, equalsIgnoreCase, compareTo and regionMatches.
(Part 1 of 5.)

```
14 // test for equality
15 if (s1.equals("hello")) { // true
16     System.out.println("s1 equals \"hello\"");
17 }
18 else {
19     System.out.println("s1 does not equal \"hello\"");
20 }
21
22 // test for equality with ==
23 if (s1 == "hello") { // false; they are not the same object
24     System.out.println("s1 is the same object as \"hello\"");
25 }
26 else {
27     System.out.println("s1 is not the same object as \"hello\"");
28 }
29
```

Fig. 14.3 | String methods `equals`, `equalsIgnoreCase`, `compareTo` and `regionMatches`.
(Part 2 of 5.)

COMMON PROGRAMMING ERROR



“==“ ile referansları karşılaştırmak mantık hatalarına yol açabilir, çünkü “==“ iki nesnenin aynı içeriğe sahip olup olmadığını değil, aynı nesneye atıfta bulunup bulunmadıklarını belirlemek için referansları karşılaştırır.

Aynı değerleri içeren iki AYRI nesne == ile karşılaştırıldığında sonuç yanlış olacaktır. Aynı içeriğe sahip olup olmadıklarını belirlemek için nesneleri karşılaştırırken, equals() kullanın.

```
30 // test for equality (ignore case)
31 if (s3.equalsIgnoreCase(s4)) { // true
32     System.out.printf("%s equals %s with case ignored%n", s3, s4);
33 }
34 else {
35     System.out.println("s3 does not equal s4");
36 }
37
38 // test compareTo
39 System.out.printf(
40     "%ns1.compareTo(s2) is %d", s1.compareTo(s2));
41 System.out.printf(
42     "%ns2.compareTo(s1) is %d", s2.compareTo(s1));
43 System.out.printf(
44     "%ns1.compareTo(s1) is %d", s1.compareTo(s1));
45 System.out.printf(
46     "%ns3.compareTo(s4) is %d", s3.compareTo(s4));
47 System.out.printf(
48     "%ns4.compareTo(s3) is %d%n%n", s4.compareTo(s3));
```

Fig. 14.3 | String methods equals, equalsIgnoreCase, compareTo and regionMatches.
(Part 3 of 5.)

```
49
50 // test regionMatches (case sensitive)
51 if (s3.regionMatches(0, s4, 0, 5)) {
52     System.out.println("First 5 characters of s3 and s4 match");
53 }
54 else {
55     System.out.println(
56         "First 5 characters of s3 and s4 do not match");
57 }
58
59 // test regionMatches (ignore case)
60 if (s3.regionMatches(true, 0, s4, 0, 5)) {
61     System.out.println(
62         "First 5 characters of s3 and s4 match with case ignored");
63 }
64 else {
65     System.out.println(
66         "First 5 characters of s3 and s4 do not match");
67 }
68 }
69 }
```

Fig. 14.3 | String methods equals, equalsIgnoreCase, compareTo and regionMatches.
(Part 4 of 5.)


```
s1 = hello
s2 = goodbye
s3 = Happy Birthday
s4 = happy birthday

s1 equals "hello"
s1 is not the same object as "hello"
Happy Birthday equals happy birthday with case ignored

s1.compareTo(s2) is 1
s2.compareTo(s1) is -1
s1.compareTo(s1) is 0
s3.compareTo(s4) is -32
s4.compareTo(s3) is 32

First 5 characters of s3 and s4 do not match
First 5 characters of s3 and s4 match with case ignored
```

Fig. 14.3 | String methods equals, equalsIgnoreCase, compareTo and regionMatches.
(Part 5 of 5.)



▶ String başlama ve bitiş indekslerini inceleme

▶ `startsWith()`

▶ `endsWith()`

```
1 // Fig. 14.4: StringStartEnd.java
2 // String methods startsWith and endsWith.
3
4 public class StringStartEnd {
5     public static void main(String[] args) {
6         String[] strings = {"started", "starting", "ended", "ending"};
7
8         // test method startsWith
9         for (String string : strings) {
10             if (string.startsWith("st")) {
11                 System.out.printf("\"%s\" starts with \"st\"%n", string);
12             }
13         }
14
15         System.out.println();
16
```

Fig. 14.4 | String methods startsWith and endsWith. (Part 1 of 3.)

```
17 // test method startsWith starting from position 2 of string
18 for (String string : strings) {
19     if (string.startsWith("art", 2)) {
20         System.out.printf(
21             "\"%s\" starts with \"art\" at position 2%n", string);
22     }
23 }
24
25 System.out.println();
26
27 // test method endsWith
28 for (String string : strings) {
29     if (string.endsWith("ed")) {
30         System.out.printf("\"%s\" ends with \"ed\"%n", string);
31     }
32 }
33 }
34 }
```

Fig. 14.4 | String methods startsWith and endsWith. (Part 2 of 3.)


```
"started" starts with "st"  
"starting" starts with "st"
```

```
"started" starts with "art" at position 2  
"starting" starts with "art" at position 2
```

```
"started" ends with "ed"  
"ended" ends with "ed"
```

Fig. 14.4 | String methods `startsWith` and `endsWith`. (Part 3 of 3.)



▶ String indekslerini inceleme

▶ `indexOf()`

▶ `lastIndexOf()`

```
1 // Fig. 14.5: StringIndexMethods.java
2 // String searching methods indexOf and lastIndexOf.
3
4 public class StringIndexMethods {
5     public static void main(String[] args) {
6         String letters = "abcdefghijklmabcdefghijklm";
7
8         // test indexOf to locate a character in a string
9         System.out.printf(
10             "'c' is located at index %d\n", letters.indexOf('c'));
11         System.out.printf(
12             "'a' is located at index %d\n", letters.indexOf('a', 1));
13         System.out.printf(
14             "'$' is located at index %d\n\n", letters.indexOf('$'));
15
```

Fig. 14.5 | String-searching methods indexOf and lastIndexOf. (Part 1 of 4.)

```
16 // test lastIndexOf to find a character in a string
17 System.out.printf("Last 'c' is located at index %d\n",
18     letters.lastIndexOf('c'));
19 System.out.printf("Last 'a' is located at index %d\n",
20     letters.lastIndexOf('a', 25));
21 System.out.printf("Last '$' is located at index %d\n\n",
22     letters.lastIndexOf('$'));
23
24 // test indexOf to locate a substring in a string
25 System.out.printf("\"def\" is located at index %d\n",
26     letters.indexOf("def"));
27 System.out.printf("\"def\" is located at index %d\n",
28     letters.indexOf("def", 7));
29 System.out.printf("\"hello\" is located at index %d\n\n",
30     letters.indexOf("hello"));
```

Fig. 14.5 | String-searching methods `indexOf` and `lastIndexOf`. (Part 2 of 4.)

```
31
32 // test lastIndexOf to find a substring in a string
33 System.out.printf("Last \"def\" is located at index %d%n",
34     letters.lastIndexOf("def"));
35 System.out.printf("Last \"def\" is located at index %d%n",
36     letters.lastIndexOf("def", 25));
37 System.out.printf("Last \"hello\" is located at index %d%n",
38     letters.lastIndexOf("hello"));
39 }
40 }
```

Fig. 14.5 | String-searching methods indexOf and lastIndexOf. (Part 3 of 4.)

```
'c' is located at index 2  
'a' is located at index 13  
'$' is located at index -1
```

```
Last 'c' is located at index 15  
Last 'a' is located at index 13  
Last '$' is located at index -1
```

```
"def" is located at index 3  
"def" is located at index 16  
"hello" is located at index -1
```

```
Last "def" is located at index 16  
Last "def" is located at index 16  
Last "hello" is located at index -1
```

Fig. 14.5 | String-searching methods `indexOf` and `lastIndexOf`. (Part 4 of 4.)



- ▶ String ile alt string (substring) elde etme
 - ▶ `substring()`
- ▶ String birleştirme
 - ▶ `concat()` (“+” operatöründen farklı olarak yeni bir string oluşturulur.)


```
1 // Fig. 14.6: SubString.java
2 // String class substring methods.
3
4 public class SubString {
5     public static void main(String[] args) {
6         String letters = "abcdefghijklmabcdefghijklm";
7
8         // test substring methods
9         System.out.printf("Substring from index 20 to end is \"%s\"%n",
10             letters.substring(20));
11         System.out.printf("%s \"%s\"%n",
12             "Substring from index 3 up to, but not including, 6 is",
13             letters.substring(3, 6));
14     }
15 }
```

Substring from index 20 to end is "hijklm"
Substring from index 3 up to, but not including, 6 is "def"

Fig. 14.6 | String class substring methods.

```
1 // Fig. 14.7: StringConcatenation.java
2 // String method concat.
3
4 public class StringConcatenation {
5     public static void main(String[] args) {
6         String s1 = "Happy ";
7         String s2 = "Birthday";
8
9         System.out.printf("s1 = %s\ns2 = %s\n\n", s1, s2);
10        System.out.printf(
11            "Result of s1.concat(s2) = %s\n", s1.concat(s2));
12        System.out.printf("s1 after concatenation = %s\n", s1);
13    }
14 }
```

```
s1 = Happy
s2 = Birthday
```

```
Result of s1.concat(s2) = Happy Birthday
s1 after concatenation = Happy
```

Fig. 14.7 | String method concat.



- ▶ Yer değiştirme
 - ▶ `replace()`
- ▶ Büyük-küçük harf değişimi
 - ▶ `toUpperCase()`
 - ▶ `toLowerCase()`
- ▶ Boşlukları silme
 - ▶ `trim()`
- ▶ Diziye çevirme
 - ▶ `toCharArray()`

```
1  // Fig. 14.8: StringMiscellaneous2.java
2  // String methods replace, toLowerCase, toUpperCase, trim and toCharArray.
3
4  public class StringMiscellaneous2 {
5      public static void main(String[] args) {
6          String s1 = "hello";
7          String s2 = "GOODBYE";
8          String s3 = "    spaces    ";
9
10         System.out.printf("s1 = %s\ns2 = %s\ns3 = %s\n\n", s1, s2, s3);
11
12         // test method replace
13         System.out.printf(
14             "Replace 'l' with 'L' in s1: %s\n\n", s1.replace('l', 'L'));
15
16         // test toLowerCase and toUpperCase
17         System.out.printf("s1.toUpperCase() = %s\n", s1.toUpperCase());
18         System.out.printf("s2.toLowerCase() = %s\n\n", s2.toLowerCase());
```

Fig. 14.8 | String methods replace, toLowerCase, toUpperCase, trim and toCharArray.
(Part 1 of 3.)

```
19
20 // test trim method
21 System.out.printf("s3 after trim = \"%s\\n\\n\", s3.trim());
22
23 // test toCharArray method
24 char[] charArray = s1.toCharArray();
25 System.out.print("s1 as a character array = ");
26
27 for (char character : charArray) {
28     System.out.print(character);
29 }
30
31 System.out.println();
32 }
33 }
```

Fig. 14.8 | String methods replace, toLowerCase, toUpperCase, trim and toCharArray.
(Part 2 of 3.)

```
s1 = hello
s2 = GOODBYE
s3 =     spaces

Replace 'l' with 'L' in s1: heLLo

s1.toUpperCase() = HELLO
s2.toLowerCase() = goodbye

s3 after trim = "spaces"

s1 as a character array = hello
```

Fig. 14.8 | String methods replace, toLowerCase, toUpperCase, trim and toCharArray.
(Part 3 of 3.)



- ▶ Her nesne toString ile string temsili elde edilir.
- ▶ Ancak primitive tipler için bu metot kullanılmaz.
- ▶ String.valueOf() burada devreye girer.

```
1 // Fig. 14.9: StringValueOf.java
2 // String valueOf methods.
3
4 public class StringValueOf {
5     public static void main(String[] args) {
6         char[] charArray = {'a', 'b', 'c', 'd', 'e', 'f'};
7         boolean booleanValue = true;
8         char characterValue = 'Z';
9         int integerValue = 7;
10        long longValue = 100000000000L; // L suffix indicates long
11        float floatValue = 2.5f; // f indicates that 2.5 is a float
12        double doubleValue = 33.333; // no suffix, double is default
13        Object objectRef = "hello"; // assign string to an Object reference
14
15        System.out.printf(
16            "char array = %s\n", String.valueOf(charArray));
17        System.out.printf("part of char array = %s\n",
18            String.valueOf(charArray, 3, 3));
```

Fig. 14.9 | String valueOf methods. (Part 1 of 3.)

```
19      System.out.printf(  
20          "boolean = %s%n", String.valueOf(booleanValue));  
21      System.out.printf(  
22          "char = %s%n", String.valueOf(characterValue));  
23      System.out.printf("int = %s%n", String.valueOf(integerValue));  
24      System.out.printf("long = %s%n", String.valueOf(longValue));  
25      System.out.printf("float = %s%n", String.valueOf(floatValue));  
26      System.out.printf(  
27          "double = %s%n", String.valueOf(doubleValue));  
28      System.out.printf("Object = %s", String.valueOf(objectRef));  
29  }  
30 }
```

Fig. 14.9 | String valueOf methods. (Part 2 of 3.)

```
char array = abcdef  
part of char array = def  
boolean = true  
char = Z  
int = 7  
long = 10000000000  
float = 2.5  
double = 33.333  
Object = hello
```

Fig. 14.9 | String valueOf methods. (Part 3 of 3.)



- ▶ Bir cümle okuduğunuzda cümleyi anlamlandıracağınız bazı simgelere ayırırsınız.
 - ▶ Kelimelere,
 - ▶ Noktalama işaretlerine, vb.
- ▶ Derleyiciler de bu şekilde davranırlar.
 - ▶ Bir ifadeyi; anahtar kelimeler, operatörler, diğer programlama terimlerine ayırırlar.
- ▶ String split() metodu da, string değişkenlere token lara ayırır.
 - ▶ Bunu yaparken farklı ayırıcılar (delimiter) kullanır. (Boşluk, tab, yeniSatır, virgül, vb.)

```
1 // Fig. 14.18: TokenTest.java
2 // Tokenizing with String method split
3 import java.util.Scanner;
4
5 public class TokenTest {
6     // execute application
7     public static void main(String[] args) {
8         // get sentence
9         Scanner scanner = new Scanner(System.in);
10        System.out.println("Enter a sentence and press Enter");
11        String sentence = scanner.nextLine();
12
13        // process user sentence
14        String[] tokens = sentence.split(" ");
15        System.out.printf("Number of elements: %d\nThe tokens are:%n",
16                          tokens.length);
17
18        for (String token : tokens) {
19            System.out.println(token);
20        }
21    }
22 }
```

Fig. 14.18 | Tokenizing with String method split. (Part 1 of 2.)

```
Enter a sentence and press Enter
This is a sentence with seven tokens
Number of elements: 7
The tokens are:
This
is
a
sentence
with
seven
tokens
```

Fig. 14.18 | Tokenizing with String method `split`. (Part 2 of 2.)



- ▶ Düzenli ifade, bir başka String için eşleşen karakterler için bir “arama deseni” tanımlayan bir String dir.
- ▶ Bu tür ifadeler, girdiyi doğrulamak ve verilerin belirli bir biçimde olmasını sağlamak için kullanışlıdır. Örneğin,
 - ▶ Bir posta kodu beş basamaktan oluşmalıdır
 - ▶ Soyadı yalnızca harfler, boşluklar, kesme işaretleri ve kısa çizgiler içermelidir.
- ▶ Düzenli ifadelerin bir uygulaması, bir derleyicinin oluşturulmasını kolaylaştırmaktır.
 - ▶ Genellikle, bir programın sözdizimini doğrulamak için büyük ve karmaşık bir düzenli ifade kullanılır.
 - ▶ Program kodu normal ifadeyle eşleşmiyorsa, derleyici kodda bir sözdizimi hatası olduğunu bilir.
- ▶ String sınıfı, düzenli ifade işlemlerini gerçekleştirmek için bazı metotlar sağlar. MATCHES() metodu;
 - ▶ Düzenli ifadeyi belirten bir argüman alır ve çağrıldığı String nesnesinin içeriğiyle düzenli ifadeyi eşleştirir.
 - ▶ Eşleşmenin başarılı olup olmadığını gösteren bir boolean değer döndürür.



- ▶ Düzenli bir ifade, değişmez karakterlerden ve özel sembollerden oluşur.
- ▶ Şekil 14.19, düzenli ifadelerle kullanılabilecek önceden tanımlanmış bazı karakter sınıflarını belirtir.
- ▶ Bir karakter sınıfı, bir karakter grubunu temsil eden bir kaçış dizisidir. (Escape char)
 - ▶ “\d” herhangi bir sayısal karakterdir.
 - ▶ “\w” herhangi bir harf (büyük veya küçük harf), herhangi bir rakam veya alt çizgi karakteridir.
 - ▶ “\s” boşluk, sekme (\t), satır başı (\r), yeni satır(\n) veya form feed (\f).

Character	Matches	Character	Matches
\d	any digit	\D	any nondigit
\w	any word character	\W	any nonword character
\s	any white-space character	\S	any non-whitespace character

Fig. 14.19 | Predefined character classes.



- ▶ Düzenli bir ifade bunlarla sınıflı değildir;
 - ▶ Önceden tanımlanmış bir karakter sınıfı olmayan bir karakter kümesini eşleştirmek için köşeli parantez `[]` kullanılır.
 - ▶ Örneğin, `"[aeiou]"` kalıbı sesli harf olan tek bir karakterle eşleşir.
 - ▶ Karakter aralıkları, iki karakter arasına tire `(-)` konularak temsil edilir.
 - ▶ Örneğin, `"[A-Z]"` tek bir büyük harfle eşleşir.
 - ▶ Parantez içindeki ilk karakter `"^"` ise, ifade belirtilenler dışında herhangi bir karakteri kabul eder.
 - ▶ Ancak, `"[^Z]"`, A-Y büyük harfleriyle eşleşen `"[A-Y]"` ile aynı değildir
 - ▶ `"[^Z]"`, küçük harfler ve harf olmayanlar dahil olmak üzere büyük Z dışındaki herhangi bir karakterle eşleşir yeni satır karakteri gibi.
 - ▶ Karakter sınıflarındaki aralıklar, harflerin tamsayı değerlerine göre belirlenir.
 - ▶ Bu örnekte, `"[A-Za-z]"` tüm büyük ve küçük harflerle eşleşir.
 - ▶ `"[A-z]"` aralığı tüm harflerle eşleşir ve aynı zamanda bu karakterlerle (`[` ve `\` gibi) büyük Z ve küçük a arasında bir tamsayı değeriyle eşleşir. AYRINTILAR İÇİN ASCII İNCELEYİN.

```
1 // Fig. 14.20: ValidateInput.java
2 // Validating user information using regular expressions.
3
4 public class ValidateInput {
5     // validate first name
6     public static boolean validateFirstName(String firstName) {
7         return firstName.matches("[A-Z][a-zA-Z]*");
8     }
9
10    // validate last name
11    public static boolean validateLastName(String lastName) {
12        return lastName.matches("[a-zA-z]+(['-][a-zA-Z]+)*");
13    }
14
15    // validate address
16    public static boolean validateAddress(String address) {
17        return address.matches(
18            "\\d+\\s+([a-zA-Z]+|[a-zA-Z]+\\s[a-zA-Z]+)");
19    }
```

Fig. 14.20 | Validating user information using regular expressions. (Part 1 of 2.)

```
20
21 // validate city
22 public static boolean validateCity(String city) {
23     return city.matches("[a-zA-Z]+|[a-zA-Z]+\\s[a-zA-Z]+");
24 }
25
26 // validate state
27 public static boolean validateState(String state) {
28     return state.matches("[a-zA-Z]+|[a-zA-Z]+\\s[a-zA-Z]+");
29 }
30
31 // validate zip
32 public static boolean validateZip(String zip) {
33     return zip.matches("\\d{5}");
34 }
35
36 // validate phone
37 public static boolean validatePhone(String phone) {
38     return phone.matches("[1-9]\\d{2}-[1-9]\\d{2}-\\d{4}");
39 }
40 }
```

Fig. 14.20 | Validating user information using regular expressions. (Part 2 of 2.)

```
1 // Fig. 14.21: Validate.java
2 // Input and validate data from user using the ValidateInput class.
3 import java.util.Scanner;
4
5 public class Validate {
6     public static void main(String[] args) {
7         // get user input
8         Scanner scanner = new Scanner(System.in);
9         System.out.println("Please enter first name:");
10        String firstName = scanner.nextLine();
11        System.out.println("Please enter last name:");
12        String lastName = scanner.nextLine();
13        System.out.println("Please enter address:");
14        String address = scanner.nextLine();
15        System.out.println("Please enter city:");
16        String city = scanner.nextLine();
17        System.out.println("Please enter state:");
18        String state = scanner.nextLine();
19        System.out.println("Please enter zip:");
20        String zip = scanner.nextLine();
21        System.out.println("Please enter phone:");
22        String phone = scanner.nextLine();
```

Fig. 14.21 | Input and validate data from user using the ValidateInput class. (Part 1 of 5.)

```
23
24 // validate user input and display error message
25 System.out.printf("%nValidate Result:");
26
27 if (!ValidateInput.validateFirstName(firstName)) {
28     System.out.println("Invalid first name");
29 }
30 else if (!ValidateInput.validateLastName(lastName)) {
31     System.out.println("Invalid last name");
32 }
33 else if (!ValidateInput.validateAddress(address)) {
34     System.out.println("Invalid address");
35 }
36 else if (!ValidateInput.validateCity(city)) {
37     System.out.println("Invalid city");
38 }
```

Fig. 14.21 | Input and validate data from user using the `ValidateInput` class. (Part 2 of 5.)

```
39     else if (!ValidateInput.validateState(state)) {
40         System.out.println("Invalid state");
41     }
42     else if (!ValidateInput.validateZip(zip)) {
43         System.out.println("Invalid zip code");
44     }
45     else if (!ValidateInput.validatePhone(phone)) {
46         System.out.println("Invalid phone number");
47     }
48     else {
49         System.out.println("Valid input. Thank you.");
50     }
51 }
52 }
```

Fig. 14.21 | Input and validate data from user using the `ValidateInput` class. (Part 3 of 5.)

Please enter first name:

Jane

Please enter last name:

Doe

Please enter address:

123 Some Street

Please enter city:

Some City

Please enter state:

SS

Please enter zip:

123

Please enter phone:

123-456-7890

Validate Result:

Invalid zip code

Fig. 14.21 | Input and validate data from user using the `ValidateInput` class. (Part 4 of 5.)

Please enter first name:

Jane

Please enter last name:

Doe

Please enter address:

123 Some Street

Please enter city:

Some City

Please enter state:

SS

Please enter zip:

12345

Please enter phone:

123-456-7890

Validate Result:

Valid input. Thank you.

Fig. 14.21 | Input and validate data from user using the `ValidateInput` class. (Part 5 of 5.)



- ▶ 7. satırda, ikinci karakter sınıfından sonraki yıldız işareti, herhangi bir sayıda harfin eşleştirilebileceğini belirtir. Genel olarak, düzenli ifade operatörü "*" normal bir ifadede görüldüğünde,
 - ▶ uygulama "*" den hemen önceki alt ifadenin sıfır veya daha fazla tekrarını eşleştirmeye çalışır.
 - ▶ "+" operatörü, "+"dan hemen önceki alt ifadenin bir veya daha fazla tekrarını eşleştirmeye çalışır.
 - ▶ Böylece hem "A*" hem de "A+", "AAA" veya "A" ile eşleşir, ancak yalnızca "A*" boş bir dizeyle eşleşir.



- ▶ 33. satırı, ilk karakter sınıfı, herhangi bir rakamla bir veya daha fazla kez (`\\d+`) eşleşir.
- ▶ iki `\` karakteri kullanılır, çünkü `\` normalde bir dizgede bir kaçış dizisi başlatır.
- ▶ Yani bir Dize'deki `\\d`, normal ifade modelini `\d` temsil eder.
- ▶ Sonra bir veya daha fazla boşluk karakteriyle (`\\s+`) eşleştiririz.
- ▶ `"|"` karakteri ifadeyi soluyla veya sağıyla eşleştirir.
- ▶ Örneğin, `"Merhaba (John|Jane)"`, hem `"Merhaba John"` hem de `"Merhaba Jane"` ile eşleşir.
- ▶ Bu örnekte, `|` tek bir sözcükle eşleşir ve sağ taraf, herhangi bir miktarda boşlukla ayrılmış iki sözcükle eşleşir.
- ▶ Bu nedenle adres, bir veya iki kelimeden sonra bir sayı içermelidir.
 - ▶ bu örnekte hem `"10 Broadway"` hem de `"10 Main Street"` geçerli adreslerdir.
- ▶ Şehir (Şekil 14.20'nin 22-24. satırları) ve durum (Şekil 14.20'nin 27-29. satırları) yöntemleri de en az bir karakterden oluşan herhangi bir kelimeyle veya alternatif olarak, eğer sözcükler tek bir boşlukla ayrılmış, böylece hem `Waltham` hem de `West Newton` eşleşecektir.



- ▶ Posta kodu (Şekil 14.20'deki 33. satır) bir rakamla beş kez eşleşir. Bu normal ifade, rakam karakter sınıfını ve parantezler arasında 5 rakamı olan bir niceleyiciyi kullanır.
- ▶ Telefon numarası (Şekil 14.20'deki 38. satır)
 - ▶ üç rakamla (birincisi sıfır olamaz),
 - ▶ ardından bir tire ve ardından üç rakam daha (yine ilki sıfır olamaz) ve
 - ▶ ardından dört rakamla eşleşir.

Quantifier	Matches
*	Matches zero or more occurrences of the pattern.
+	Matches one or more occurrences of the pattern.
?	Matches zero or one occurrences of the pattern.
{ <i>n</i> }	Matches exactly <i>n</i> occurrences.
{ <i>n</i> , }	Matches at least <i>n</i> occurrences.
{ <i>n</i> , <i>m</i> }	Matches between <i>n</i> and <i>m</i> (inclusive) occurrences.

Fig. 14.22 | Quantifiers used in regular expressions.

```
1 // Fig. 14.23: RegexSubstitution.java
2 // String methods replaceFirst, replaceAll and split.
3 import java.util.Arrays;
4
5 public class RegexSubstitution {
6     public static void main(String[] args) {
7         String firstString = "This sentence ends in 5 stars *****";
8         String secondString = "1, 2, 3, 4, 5, 6, 7, 8";
9
10        System.out.printf("Original String 1: %s\n", firstString);
11
12        // replace '*' with '^'
13        firstString = firstString.replaceAll("\\*", "^");
14
15        System.out.printf("^ substituted for *: %s\n", firstString);
16
17        // replace 'stars' with 'carets'
18        firstString = firstString.replaceAll("stars", "carets");
19
20        System.out.printf(
21            "\"carets\" substituted for \"stars\": %s\n", firstString);
```

Fig. 14.23 | String methods replaceFirst, replaceAll and split. (Part 1 of 3.)

```
22
23 // replace words with 'word'
24 System.out.printf("Every word replaced by \"word\": %s%n%n",
25     firstString.replaceAll("\\w+", "word"));
26
27 System.out.printf("Original String 2: %s%n", secondString);
28
29 // replace first three digits with 'digit'
30 for (int i = 0; i < 3; i++) {
31     secondString = secondString.replaceFirst("\\d", "digit");
32 }
33
34 System.out.printf(
35     "First 3 digits replaced by \"digit\" : %s%n", secondString);
36
37 System.out.print("String split at commas: ");
38 String[] results = secondString.split(",\\s*"); // split on commas
39 System.out.println(Arrays.toString(results));
40 }
41 }
```

Fig. 14.23 | String methods `replaceFirst`, `replaceAll` and `split`. (Part 2 of 3.)

```
Original String 1: This sentence ends in 5 stars *****
^ substituted for *: This sentence ends in 5 stars ^^^^^
"carets" substituted for "stars": This sentence ends in 5 carets ^^^^^
Every word replaced by "word": word word word word word word ^^^^^

Original String 2: 1, 2, 3, 4, 5, 6, 7, 8
First 3 digits replaced by "digit" : digit, digit, digit, 4, 5, 6, 7, 8
String split at commas: [digit, digit, digit, 4, 5, 6, 7, 8]
```

Fig. 14.23 | String methods `replaceFirst`, `replaceAll` and `split`. (Part 3 of 3.)



- ▶ Düzenli bir ifade yalnızca bir kez kullanılacaksa, `matches()` kullanılabilir.
- ▶ Düzenli bir ifade bir kereden fazla kullanılacaksa (örneğin bir döngüde), o düzenli ifade için belirli bir Pattern nesnesi oluşturmak üzere `compile()` metodunu kullanmak daha verimlidir.
- ▶ Bu metod, düzenli ifadeyi temsil eden bir String alır ve daha sonra `matcher()` yöntem çağırarak için kullanılacak yeni bir Pattern nesnesi döndürür.
- ▶ `Matcher()`, `matches()` aynı görevi gerçekleştiren, ancak hiçbir argüman almayan yöntem eşleşmeleri sağlar; arama modeli ve arama nesnesi, Eşleştirici nesnesinde kapsülendir. Class `Matcher` `find`, `lookAt`, `replaceFirst` ve `replaceAll` dahil olmak üzere başka yöntemler sağlar.
- ▶ Şekil 14.24, düzenli ifadeler kullanan basit bir örnek sunar. Bu program doğum günlerini normal bir ifadeyle eşleştirir. Bu ifade yalnızca Nisan ayında olmayan ve adları "J" ile başlayan kişilere ait olan doğum günleriyle eşleşir.

```
1 // Fig. 14.24: RegexMatches.java
2 // Classes Pattern and Matcher.
3 import java.util.regex.Matcher;
4 import java.util.regex.Pattern;
5
6 public class RegexMatches {
7     public static void main(String[] args) {
8         // create regular expression
9         Pattern expression =
10             Pattern.compile("J.*\\d[0-35-9]-\\d\\d-\\d\\d");
11
12         String string1 = "Jane's Birthday is 05-12-75\n" +
13             "Dave's Birthday is 11-04-68\n" +
14             "John's Birthday is 04-28-73\n" +
15             "Joe's Birthday is 12-17-77";
```

Fig. 14.24 | Classes Pattern and Matcher. (Part 1 of 2.)

```
16
17 // match regular expression to string and print matches
18 Matcher matcher = expression.matcher(string1);
19
20 while (matcher.find()) {
21     System.out.println(matcher.group());
22 }
23 }
24 }
```

```
Jane's Birthday is 05-12-75
Joe's Birthday is 12-17-77
```

Fig. 14.24 | Classes Pattern and Matcher. (Part 2 of 2.)



- ▶ Değiştirilebilir (mutable) karakter dizileri oluşturmak için kullanılan bir sınıftır.
- ▶ Karakter dizileri üzerinde yapılan değişikliklerin daha verimli olmasını sağlar, özellikle büyük veya sık değiştirilen metinler için tercih edilir.
- ▶ StringBuilder sınıfı, String sınıfına benzer özelliklere sahiptir ancak farkı, StringBuilder'in değiştirilebilir olmasıdır.
 - ◆ **StringBuilder nesneleri üzerinde yapılan işlemler doğrudan nesneyi değiştirir**
 - ◆ **String nesneleri üzerinde yapılan işlemler yeni bir karakter dizisi oluşturur.**



► Örnek

```
public class StringBuilderExample {  
    public static void main(String[] args) {  
        StringBuilder sb = new StringBuilder(); // Boş bir StringBuilder nesnesi oluşturulur  
  
        sb.append("Java "); // Metin eklenir  
        sb.append("StringBuilder "); // Metin eklenir  
        sb.append("example."); // Metin eklenir  
  
        System.out.println(sb.toString()); // StringBuilder içeriği yazdırılır  
  
        sb.insert(5, "is a "); // Belirli bir konuma metin eklenir  
        System.out.println(sb.toString()); // StringBuilder içeriği yeniden yazdırılır  
  
        sb.delete(4, 7); // Belirli bir aralıktaki metin silinir  
        System.out.println(sb.toString()); // StringBuilder içeriği yeniden yazdırılır  
    }  
}
```



- ▶ Character sınıfı, bir karakterin temsil edilmesi için kullanılır.
- ▶ Her bir Character nesnesi, bir karakterin bir char veri tipi değeri ile ilişkilendirilmiştir.
- ▶ Karakterlerle ilgili farklı işlemleri gerçekleştirmek için bir dizi metod sağlar.
 - ▶ karakterin büyük veya küçük harf olup olmadığını kontrol etme,
 - ▶ rakam mı yoksa harf mi olduğunu belirleme,
 - ▶ karakterin sayısal değerini alma
- ▶ Bir dizi sabit değeri içerir,
 - ▶ Character.MAX_VALUE ve Character.MIN_VALUE
- ▶ **Unicode Desteği:** Java'da karakterler Unicode tabanlıdır Unicode karakterlerle çalışmak için kullanılır. Bu sayede, dünya çapındaki farklı alfabeler ve sembollerle etkileşime geçmek mümkün olur.



```
public class CharacterExample {  
    public static void main(String[] args) {  
        char myChar = 'A';  
  
        // Character sınıfının metodlarıyla işlemler  
        System.out.println("Karakter büyük harf mi? " + Character.isUpperCase(myChar));  
        System.out.println("Karakter küçük harf mi? " + Character.isLowerCase(myChar));  
        System.out.println("Karakter bir rakam mı? " + Character.isDigit(myChar));  
        System.out.println("Karakter sayısal değeri: " + Character.getNumericValue(myChar));  
  
        // Sabit karakterler  
        System.out.println("En büyük karakter değeri: " + (int) Character.MAX_VALUE);  
        System.out.println("En küçük karakter değeri: " + (int) Character.MIN_VALUE);  
    }  
}
```




```
public class CharacterExample {  
    public static void main(String[] args) {  
        String filePath = "metinDosyasi.txt";  
        char arananKarakter = 'a';  
        int karakterSayisi = 0;  
        ...  
        //CHARACTER SINIFI İLE  
        char karakter = (char) c;  
        if (Character.toLowerCase(karakter) == Character.toLowerCase(arananKarakter))  
            karakterSayisi++;  
        ...  
        //char veri tipi İLE  
        if (karakter == arananKarakter || (karakter >= 'A' && karakter <= 'Z' && karakter -  
            'A' + 'a' == arananKarakter))  
            karakterSayisi++;  
        ...  
    }  
}
```